



Instituto Tecnológico Superior de Lerdo

“Desarrollo de Aplicaciones para Dispositivos Móviles”

Unidad 3

Aplicación de Flutter

Alumna: Cristal Ruiz Lozano #222310571

Docente: Jesús Salas Marín

04/05/2026

Índice

Introducción	3
Configuración del entorno de Flutter	3
Creación del Proyecto en Flutter	3
Ejecución y prueba inicial de la aplicación	7
Mejoras visuales y diseño de la interfaz.....	12
Implementación de favoritos (lógica e interfaz)	21
Implementación de navegación y diseño responsivo	24
Creación de la página de favoritos	34
Conclusión	39

Parte 1 de 8

Introducción

En esta práctica se utiliza Flutter, un kit de desarrollo de interfaces creado por Google, que permite crear aplicaciones para dispositivos móviles, web y escritorio usando una sola base de código.

El objetivo es desarrollar una aplicación que genera nombres aleatorios, permite al usuario obtener nuevos, marcarlos como favoritos y visualizarlos en una sección independiente. Además, la app se adapta a diferentes tamaños de pantalla.

Durante la práctica se aprenden conceptos básicos de Flutter, como la creación de interfaces, manejo de interacciones del usuario, organización del código y diseño responsivo.

Parte 2 de 8

Configuración del entorno de Flutter

Para el desarrollo de la práctica se utilizó Visual Studio Code como entorno de programación. Se trabajó con un segmento de desarrollo definido, el cual corresponde al sistema operativo del equipo, permitiendo ejecutar y probar la aplicación directamente.

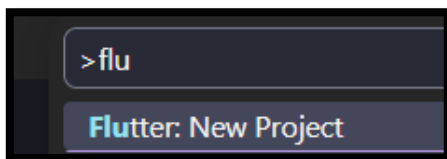
Cabe mencionar que Flutter ya se encontraba previamente instalado y configurado, por lo que se pudo comenzar directamente con el desarrollo de la aplicación.

Parte 3 de 8

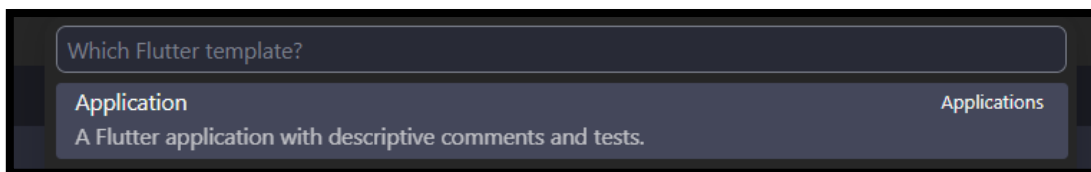
Creación del Proyecto en Flutter

1.- Se inició Visual Studio Code y se abrió la paleta de comandos presionando la combinación de teclas **Ctrl + Shift + P**. También es posible acceder presionando la tecla **F1**.

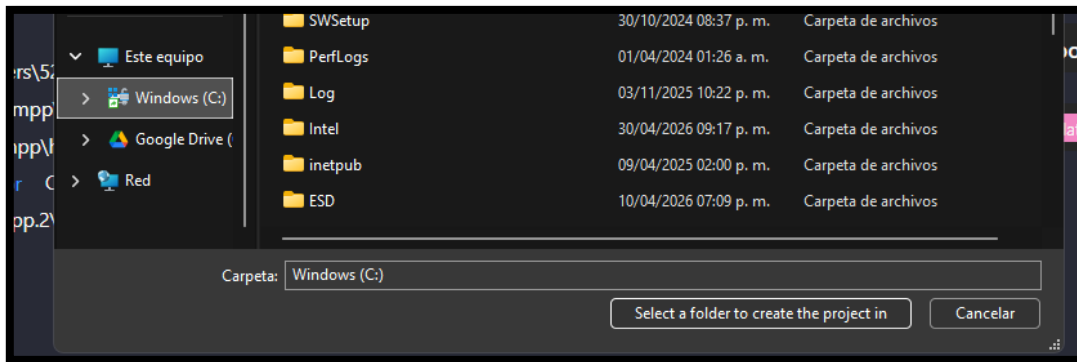
2.-Dentro de la paleta de comandos, se escribió **"Flutter"** y se seleccionó la opción **Flutter: New Project**.



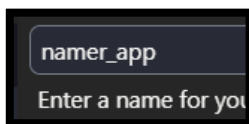
3.- Posteriormente, se eligió el tipo de proyecto **Application**.



4.- Se seleccionó como ubicación el disco local (**C:**), donde se almacenó el proyecto.

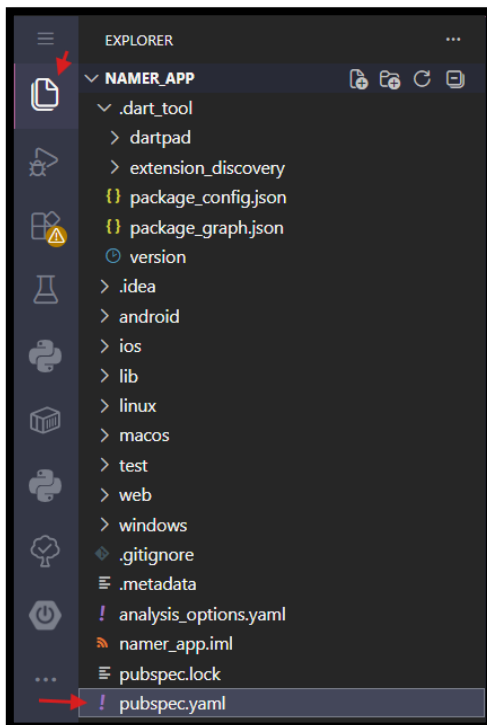


5.- Posteriormente, se asignó un nombre al proyecto, como **namer_app**.



Una vez realizado esto, Flutter generó automáticamente la estructura del proyecto dentro de la ruta correspondiente y lo abrió en el editor.

6.- Se abrió el archivo **pubspec.yaml** desde el explorador de archivos del editor:



7.- Se modificó su contenido original para agregar las dependencias necesarias, como librerías para generación de palabras y manejo de estado, quedando el código de la siguiente manera:

```

name: namer_app
description: A new Flutter project.

publish_to: 'none'

version: 0.0.1+1

environment:
  sdk: '>=2.19.4 <4.0.0'

dependencies:
  flutter:
    sdk: flutter

  english_words: ^4.0.0
  provider: ^6.0.0

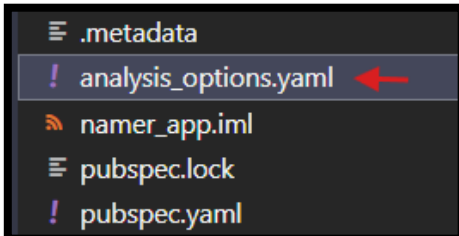
dev_dependencies:
  flutter_test:
    sdk: flutter

  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true

```

8.- Se abrió el archivo **analysis_options.yaml**:



```

.
├── .metadata
├── ! analysis_options.yaml
├── namer_app.iml
├── pubspec.lock
├── ! pubspec.yaml

```

9.-Se modificó su contenido original para establecer reglas menos estrictas durante el desarrollo, facilitando el aprendizaje, quedando el código de la siguiente manera:

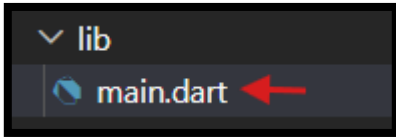
```

include: package:flutter_lints/flutter.yaml

linter:
  rules:
    prefer_const_constructors: false
    prefer_final_fields: false
    use_key_in_widget_constructors: false
    prefer_const_literals_to_create_immutables: false
    prefer_const_constructors_in_immutables: false
    avoid_print: false

```

10.- Se accedió a la carpeta **lib** y se abrió el archivo **main.dart**:



11.- Posteriormente, se reemplazó su contenido original por el código base de la aplicación, el cual incluye:

- La configuración principal de la app
- El manejo del estado
- Una interfaz básica que muestra texto y una palabra aleatoria

Quedando el código de la siguiente manera:

```
import 'package:english_words/english_words.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App',
```

```
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
      ),
      home: MyHomePage(),
    ),
  );
}

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
}
```

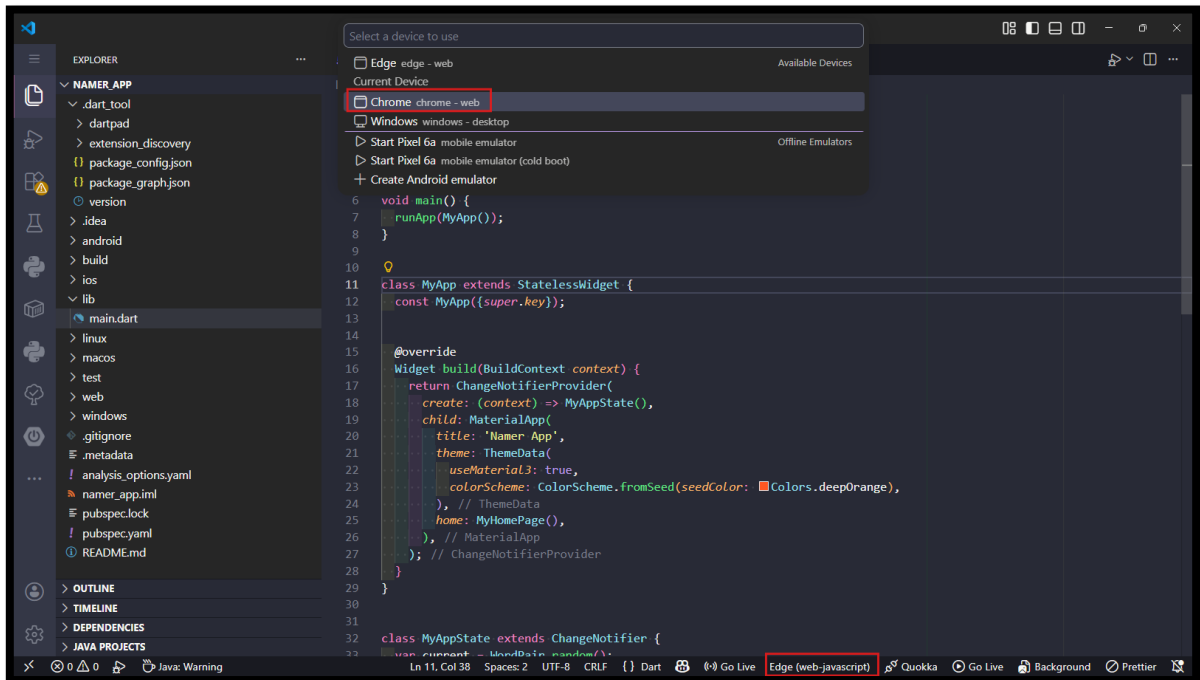
```
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch();

    return Scaffold(
      body: Column(
        children: [
          Text('A random idea:'),
          Text(appState.current.asLowerCase),
        ],
      ),
    );
  }
}
```

Parte 4 de 8

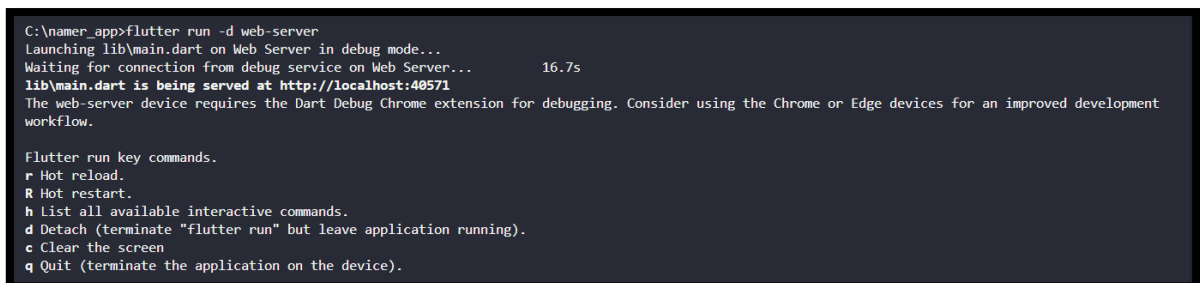
Ejecución y prueba inicial de la aplicación

- 1.- Se abrió el archivo **lib/main.dart** y se verificó el dispositivo de ejecución desde la parte inferior derecha de Visual Studio Code.
- 2.- En este caso, se seleccionó el dispositivo **Windows (x64)** como entorno de ejecución, evitando el uso de navegadores como Chrome o Edge.

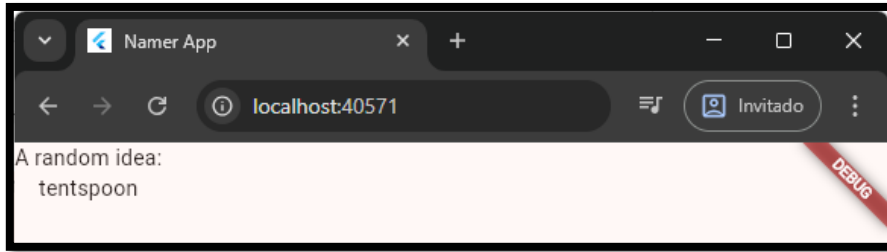


- 3.- Posteriormente, la aplicación se ejecutó desde la terminal con el comando: **flutter run -d web-server**

Este comando permitió iniciar la aplicación en modo de depuración mediante un servidor local.



Una vez ejecutado, la aplicación se abrió en el navegador mostrando la interfaz inicial con una palabra generada aleatoriamente.

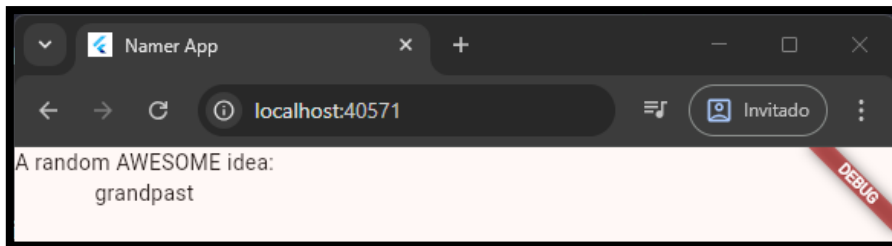


Nota: Aunque se seleccionó Windows (x64) como dispositivo, se utilizó el modo web-server, el cual ejecuta la aplicación en un navegador mediante una dirección local (localhost).

4.- Se realizó una modificación en el texto dentro del archivo **main.dart**, específicamente en el primer widget Text, agregando “AWESOME”:

```
return Scaffold(  
  body: Column(  
    children: [  
      Text('A random AWESOME idea:'),  
      Text(appState.current.asLowerCase),  
    ],  
  ),  
);
```

5.- Se guardaron los cambios (Ctrl + S) y se observó cómo la aplicación se actualizó automáticamente sin reiniciarse, manteniendo el estado actual.



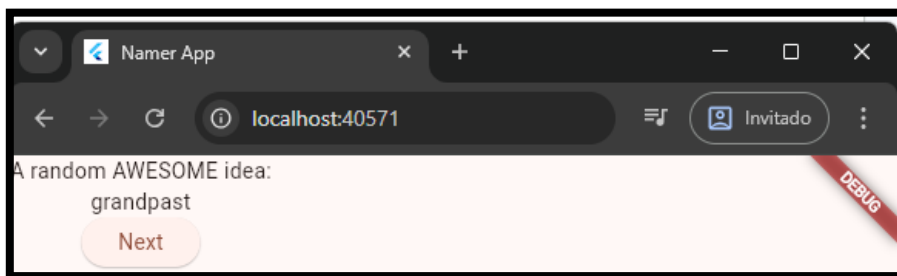
6.- Se modificó el archivo **main.dart**, agregando un botón debajo del segundo widget Text dentro del widget Column.

El botón fue implementado utilizando **ElevatedButton** y se configuró con una acción básica para verificar su funcionamiento.

El código quedó de la siguiente manera:

```
ElevatedButton(
  onPressed: () {
    print('button pressed!');
  },
  child: Text('Next'),
),
```

7.- Se guardaron los cambios (Ctrl + S) y se observó cómo la aplicación se actualizó automáticamente sin reiniciarse, mostrando el botón “Next”.



8. Análisis del código base (por secciones)

A continuación, se describe cada parte del código antes de agregar nueva funcionalidad:

8.1 Función principal (main)

Esta función es el punto de entrada de la aplicación y se encarga de ejecutar el widget principal.

```
void main() {
  runApp(MyApp());
}
```

8.2 Clase MyApp

Esta clase configura la aplicación. Define el tema visual, el nombre de la app y establece el widget principal. También integra el manejo del estado con ChangeNotifierProvider.

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App',
        theme: ThemeData(
          useMaterial3: true,
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        ),

```

```
      ),
      home: MyHomePage(),
    ),
  );
}
}
```

8.3 Clase MyAppState

Esta clase representa el estado global de la aplicación. En este punto, solo contiene una variable que almacena un par de palabras aleatorias.

```
class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
}
```

8.4 Clase MyHomePage

Esta clase define la interfaz principal de la aplicación. Utiliza el método build para construir la vista, obteniendo el estado actual mediante context.watch. La interfaz está compuesta por un Scaffold que contiene una Column, donde se muestran dos textos (uno estático y otro dinámico con la palabra aleatoria) y un botón. Este botón, en esta etapa, únicamente imprime un mensaje en la consola al ser presionado.

```
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch();

    return Scaffold(
      body: Column(
        children: [
          Text('A random AWESOME idea:'),
          Text(appState.current.asLowerCase),

```

```
          ElevatedButton(
            onPressed: () {
              print('button pressed!');
            },
            child: Text('Next'),

```

9.- Después de comprender el funcionamiento del código, se procedió a agregar funcionalidad al botón.

Se accedió a la clase **MyAppState** y se agregó el método **getNext()**, el cual genera un nuevo par de palabras aleatorias y actualiza la interfaz.

Se modificó el código, quedando de la siguiente manera:

```
class MyAppState extends ChangeNotifier {
  var current = WordPair.random();

  void getNext() {
    current = WordPair.random();
    notifyListeners();
  }
}
```

10.- Posteriormente, se modificó el botón para que ejecutara la nueva funcionalidad.

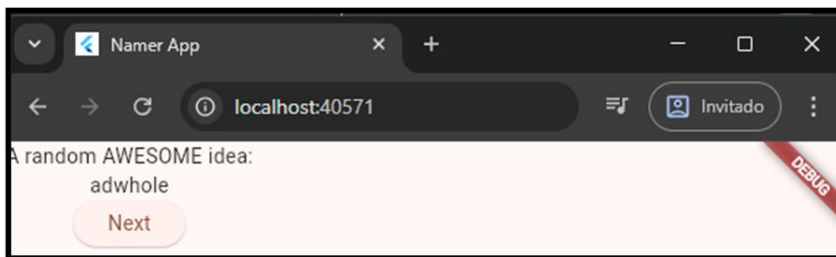
Se reemplazó la instrucción:

print('button pressed!'); por: **appState.getNext();**

De esta manera, cada vez que se presiona el botón Next, se genera una nueva combinación de palabras y la interfaz se actualiza automáticamente.

```
ElevatedButton(  
  onPressed: () {  
    appState.getNext();  
  },  
  child: Text('Next'),  
),
```

11.- Finalmente, se guardaron los cambios y se ejecutó la aplicación, verificando que al presionar el botón Next se genera automáticamente un nuevo par de palabras en pantalla.



Parte 5 de 8

Mejoras visuales y diseño de la interfaz

En esta etapa se realizaron cambios visuales para mejorar la apariencia de la aplicación, ya que inicialmente la interfaz era muy simple, con texto pequeño y sin diseño atractivo.

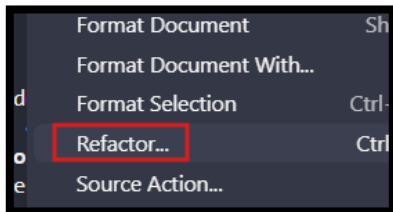
1.- Antes de realizar las mejoras visuales, se modificó la forma de acceder a la palabra generada para hacer el código más claro. Inicialmente se utilizaba directamente `Text(appState.current.asLowerCase)`, pero posteriormente se creó una variable intermedia con `var pair = appState.current;` y se reemplazó por `Text(pair.asLowerCase)`.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current; ←  
  
    return Scaffold(  
      body: Column(  
        children: [  
          Text('A random AWESOME idea:'),  
          Text(pair.asLowerCase), ←  
        ],  
      ),  
    );  
  }  
}
```

2.- Se seleccionó específicamente el widget Text dentro del código, el cual era el encargado de mostrar la palabra generada.

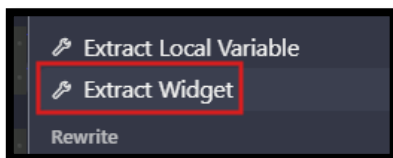
```
children: [
  Text('A random AWESOME idea:'),
  Text(pair.asLowerCase),
  ElevatedButton(
```

3.- Sobre este elemento se realizó clic derecho y se accedió al menú de opciones, donde se eligió la opción Refactor.

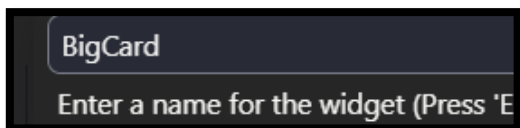


```
Format Document      Sh
Format Document With...
Format Selection      Ctrl-
Refactor...          Ctrl-
Source Action...
```

4.- Posteriormente, se seleccionó la opción **Extract Widget** y se asignó el nombre **BigCard**.



```
Extract Local Variable
Extract Widget
Rewrite
```



```
BigCard
Enter a name for the widget (Press 'E' to edit)
```

```
return Scaffold(
  body: Column(
    children: [
      Text('A random AWESOME idea:'),
      BigCard(pair: pair), ←
      ElevatedButton(
        onPressed: () {
```

Como resultado, Flutter creó automáticamente una nueva clase con ese nombre al final del archivo, moviendo ahí el código seleccionado sin afectar el funcionamiento de la aplicación.

```
class BigCard extends StatelessWidget {
  const BigCard({
    super.key,
    required this.pair,
  });

  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Text(pair.asLowerCase);
  }
}
```

5.- Después de esto, dentro de la clase BigCard, se volvió a seleccionar el widget **Text(pair.asLowerCase)** y nuevamente se utilizó la opción de **Refactor**, pero en esta ocasión se eligió **Wrap with Padding**.

Esto permitió agregar un espacio alrededor del texto para mejorar su presentación visual, aumentando posteriormente el valor del padding para que el contenido tuviera mayor separación.

```
@override
Widget build(BuildContext context) {
  return Text(pair.asLowerCase);
}
```

```
Format Document With...
Format Selection          Ctrl+K Ctrl+F
Refactor...               Ctrl+Shift+R
Source Action...         ide
```

```
OLE  TERMINAL
  ◯ Wrap with Expanded
  ◯ Wrap with Flexible
  ◯ Wrap with Padding
  ◯ Wrap with SizedBox
```

Quedando de la siguiente manera:

```
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.all(20),
    child: Text(pair.asLowerCase),
  );
}
```

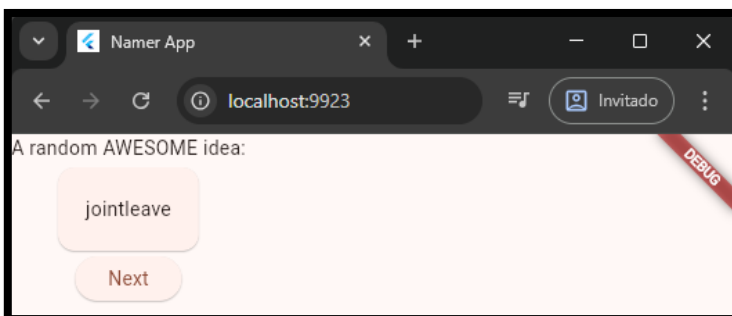
6.- A continuación, se seleccionó el widget **Padding** recién creado y, utilizando otra vez la opción **Refactor**, se eligió **Wrap with widget...** para envolverlo dentro de un widget **Card**. Esto permitió que el contenido adoptara una apariencia de tarjeta, haciéndolo más visible y atractivo dentro de la interfaz.



Quedando de la siguiente manera:

```
@override
Widget build(BuildContext context) {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(pair.asLowerCase),
    ),
  );
}
```

Esto envuelve el widget **Padding**, y en consecuencia también el **Text**, dentro de un widget **Card**, integrándolos en una sola estructura visual:

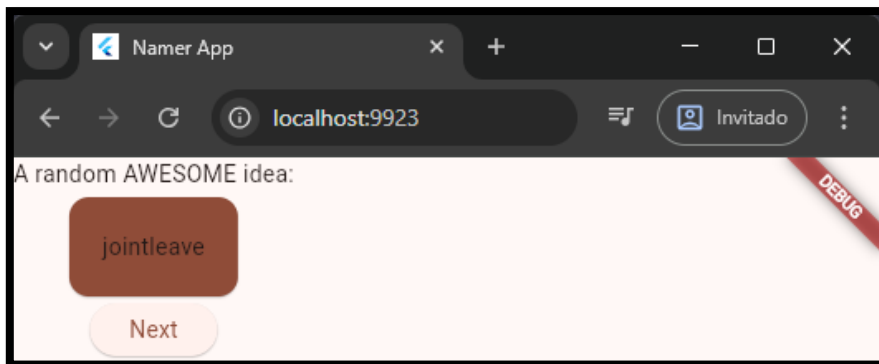


7.- Posteriormente, se trabajó en la mejora del diseño aplicando estilos mediante el tema global de la aplicación. Con el objetivo de hacer que la tarjeta resaltara más, se modificó el método `build()` de la clase `BigCard`, utilizando `Theme.of(context)` para obtener la configuración visual actual de la app. A partir de esto, se asignó el color principal del esquema (`colorScheme.primary`) como fondo de la tarjeta.

```
@override
Widget build(BuildContext context) {
  final theme = Theme.of(context); ←

  return Card(
    color: theme.colorScheme.primary, ←
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(pair.asLowerCase),
    ),
  );
}
```

Logrando así una apariencia más llamativa y coherente con el diseño general. Este enfoque permite que los colores de la aplicación se mantengan uniformes y puedan modificarse fácilmente desde un solo lugar, específicamente en la configuración del tema dentro de la clase principal.



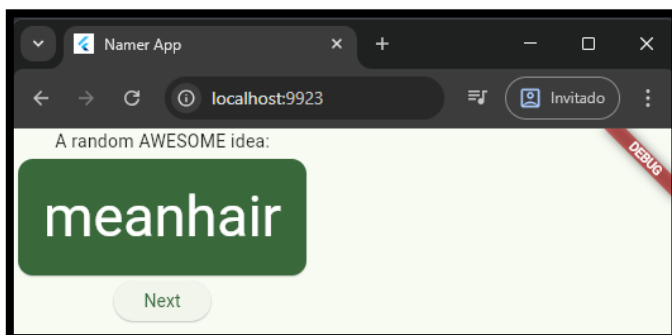
La clase `Colors` de Flutter permite acceder fácilmente a una paleta de colores escogidos a mano, como `Colors.deepOrange` o `Colors.red`



8.- Se mejoró la apariencia del texto, ya que inicialmente era pequeño y poco visible. Para ello, se definió un estilo utilizando el tema de la aplicación, aumentando el tamaño del texto y ajustando su color para lograr un mejor contraste con el fondo. Posteriormente, este estilo fue aplicado al texto principal, haciéndolo más legible y visualmente atractivo.

```
Widget build(BuildContext context) {  
  final theme = Theme.of(context);  
  final style = theme.textTheme.displayMedium!.copyWith(  
    color: theme.colorScheme.onPrimary,  
  );  
  return Card(  
    color: theme.colorScheme.primary,  
    child: Padding(  
      padding: const EdgeInsets.all(20),  
      child: Text(pair.asLowerCase, style: style),  
    ),  
  );  
}
```

La aplicación presenta un diseño más atractivo, con el texto principal más grande, mejor contraste y centrado dentro de una tarjeta de color:



Se mejoró la accesibilidad de la aplicación para facilitar su uso con lectores de pantalla. Aunque Flutter ya ofrece soporte por defecto, se detectó que algunas palabras generadas podían pronunciarse incorrectamente.

9.- Para solucionar esto, se agregó la propiedad **semanticsLabel** al widget de texto, permitiendo definir una forma más clara de lectura separando las palabras. De esta manera, la aplicación mantiene el mismo aspecto visual, pero mejora la experiencia para usuarios con discapacidad visual.

```

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );
  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(
        pair.asLowerCase,
        style: style,
        semanticsLabel: "${pair.first} ${pair.second}",
      ),
    ),
  );
}

```

10.- Posteriormente, se ajustó la posición de los elementos en pantalla, ya que inicialmente se encontraban alineados en la parte superior. Para ello, se configuró la alineación de la Column para centrar su contenido de forma vertical.

Además, se encapsuló la Column dentro de un widget Center, logrando que toda la interfaz quedara centrada también de manera horizontal. Con esto, la aplicación presenta una distribución más equilibrada y estética.

```

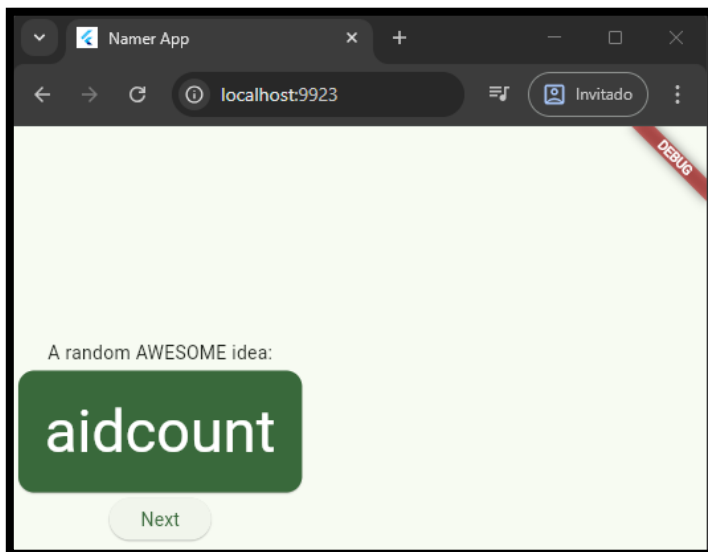
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    return Scaffold(
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text('A random AWESOME idea:'),
          BigCard(pair: pair),

          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
    );
  }
}

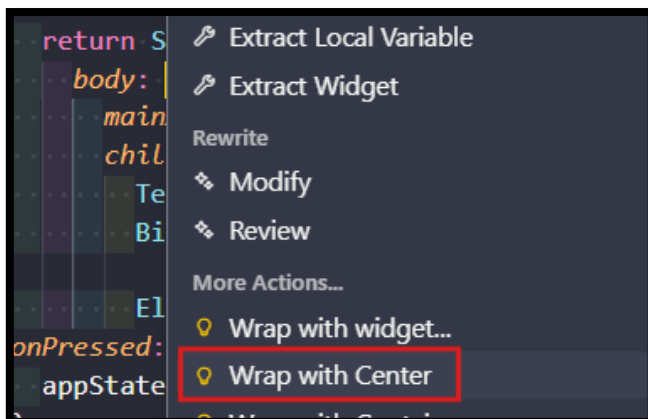
```

Aunque los elementos dentro de la Column ya se encontraban centrados horizontalmente, se observó que la estructura completa no ocupaba todo el ancho de la pantalla, por lo que no estaba completamente centrada dentro del Scaffold.



11.- Para solucionar esto, se seleccionó el widget **Column** y se utilizó la opción **Refactor** → **Wrap with Center**, lo que permitió centrar toda la estructura de la interfaz en pantalla, tanto vertical como horizontalmente.

```
return Scaffold(  
  body: Column(  
    mainAxisAlignment: MainAxis
```

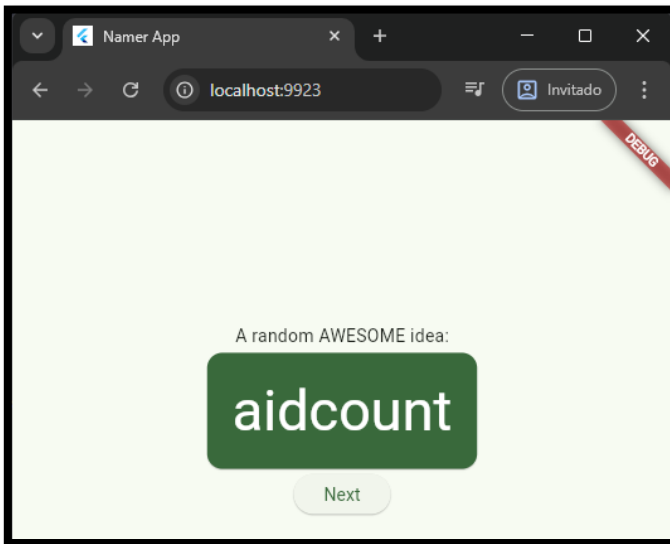


Quedando de la siguiente manera:

```
return Scaffold(  
  body: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  

```

Como resultado, la interfaz ahora se muestra completamente centrada en pantalla, logrando una mejor presentación visual.



Cabe mencionar que no se utilizó la herramienta Widget Inspector, ya que la ejecución se realizó en Windows (x64); sin embargo, el ajuste se hizo directamente mediante refactorización del código.

12.- Se realizaron mejoras visuales adicionales para optimizar la interfaz. Se eliminó el texto descriptivo superior, ya que dejó de ser necesario:

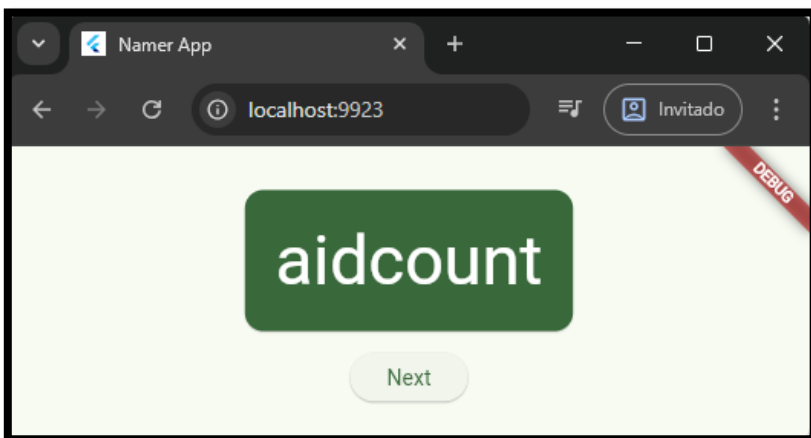
```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current;  
  
    return Scaffold(  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            Text('A random AWESOME idea:'),  
            BigCard(pair: pair),  
  
            ElevatedButton(  
              onPressed: () {  
                appState.getNext();  
              },  
              child: Text('Next')  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

13.- Se agregó un espacio entre la tarjeta y el botón utilizando un `SizeBox`, lo que permitió una mejor separación y organización de los elementos.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch();  
    var pair = appState.current;  
  
    return Scaffold(  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            BigCard(pair: pair),  
            SizeBox(height: 10), ←  
            ElevatedButton(  

```

La aplicación ahora presenta una interfaz completamente centrada, más limpia y con mejor distribución de los elementos, logrando un diseño más estético y funcional.



Parte 6 de 8

Implementación de favoritos (lógica e interfaz)

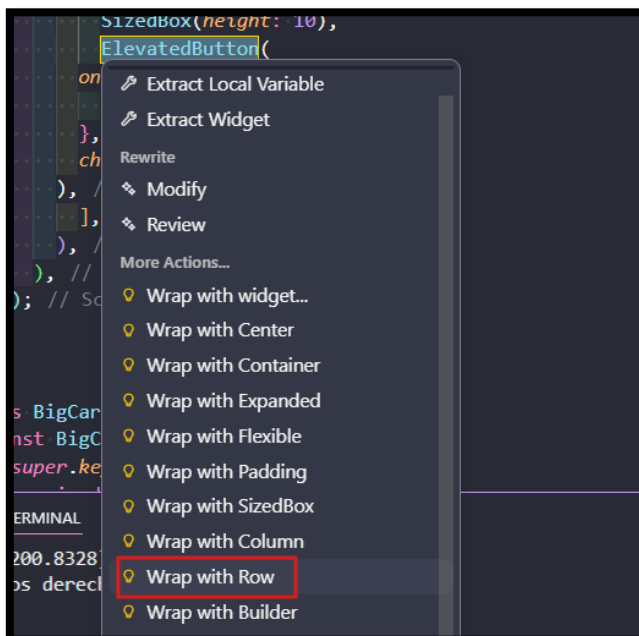
En esta etapa se agregó funcionalidad a la aplicación, permitiendo guardar las palabras generadas como favoritas, evitando que se pierdan al presionar el botón Next.

1.- Se accedió a la clase **MyAppState**, donde se agregó una lista llamada **favorites** para almacenar los pares de palabras seleccionados. Además, se implementó el método **toggleFavorite()**, el cual permite agregar o eliminar un elemento de la lista según

corresponda. También se utilizó **notifyListeners()** para actualizar la interfaz automáticamente.

```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  
  void getNext() {  
    current = WordPair.random();  
    notifyListeners();  
  }  
  
  var favorites = <WordPair>[];  
  
  void toggleFavorite() {  
    if (favorites.contains(current)) {  
      favorites.remove(current);  
    } else {  
      favorites.add(current);  
    }  
    notifyListeners();  
  }  
}
```

2.- Una vez implementada la lógica, se procedió a modificar la interfaz. Para ello, se seleccionó específicamente el widget **ElevatedButton** (correspondiente al botón Next) y se utilizó la opción **Refactor → Wrap with Row**, con el objetivo de poder agregar otro botón en la misma línea.



3.- Posteriormente, se ajustó la configuración de la **Row** para evitar que ocupara todo el ancho disponible. Para ello, se agregó la propiedad:

mainAxisSize: MainAxisSize.min,

```
my: Center(  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  
      BigCard(pair: pair),  
      SizedBox(height: 10),  
      Row(  
        mainAxisAlignment: MainAxisSize.min,  
        children: [  
          ElevatedButton(  
            onPressed: () {  
              appState.getNext();  
            },  
            child: Text('Next'),  
          ), // ElevatedButton  
        ],  
      ),  
    ],  
  ),  
),
```

4.- Se agregó una variable para controlar dinámicamente el ícono del botón dependiendo del estado de la palabra en favoritos. Para ello, se utilizó la siguiente lógica:

Con esto, el ícono cambia automáticamente entre favorito lleno o vacío según corresponda.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current;  
  
    IconData icon;  
    if (appState.favorites.contains(pair)) {  
      icon = Icons.favorite;  
    } else {  
      icon = Icons.favorite_border;  
    }  
  
    return Scaffold(  
      body: Center(  
        child: Row(  
          mainAxisAlignment: MainAxisAlignment.spaceBetween,  
          children: [  
            Text(pair),  
            ElevatedButton(  
              onPressed: () {  
                appState.toggleFavorite(pair);  
              },  
              child: Row(  
                mainAxisAlignment: MainAxisAlignment.spaceBetween,  
                children: [  
                  icon,  
                  Text('Favoritos'),  
                ],  
              ),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

5.- Posteriormente, se implementó el botón de favoritos utilizando **ElevatedButton.icon()**, el cual permite mostrar un ícono junto con el texto. Este botón fue conectado al método **toggleFavorite()** para agregar o quitar elementos de la lista de favoritos.

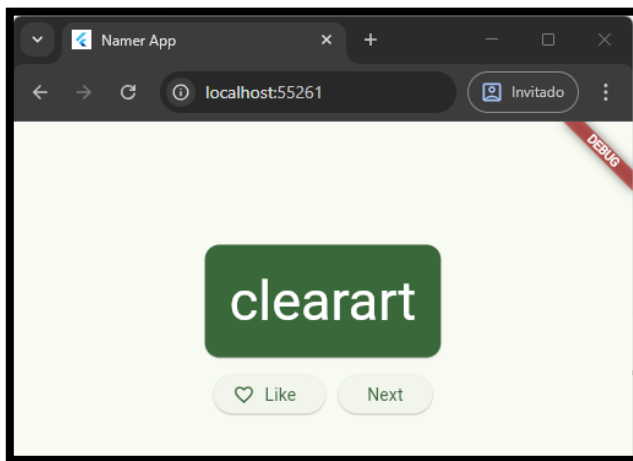
Además, se integró el ícono dinámico definido anteriormente y se agregó un **SizedBox** para mantener la separación entre los botones:

```

child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    BigCard(pair: pair),
    SizedBox(height: 10),
    Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        ElevatedButton.icon(
          onPressed: () {
            appState.toggleFavorite();
          },
          icon: Icon(icon),
          label: Text('Like'),
        ),
        SizedBox(width: 10),
        ElevatedButton(
          onPressed: () {

```

La aplicación ahora permite marcar palabras como favoritas mediante el botón Like, mejorando la interacción del usuario y evitando que se pierdan las opciones generadas.



Parte 7 de 8

Implementación de navegación y diseño responsivo

En esta etapa se implementó la navegación dentro de la aplicación, permitiendo separar la pantalla principal de una futura sección de favoritos. Para ello, se introdujo el uso de un `StatefulWidget` y un riel de navegación.

1.- Se eliminó el contenido original de **MyHomePage**:

```

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.favorite),

```

```

                  label: Text('Favorites'),
                ),
              ],
              selectedIndex: 0,
              onDestinationSelected: (value) {
                print('selected: $value');
              },
            ),
          ),
          Expanded(
            child: Container(
              color: Theme.of(context).colorScheme.primaryContainer,
              child: GeneratorPage(),
            ),
          ),
        ],
      ),
    );
  }
}

```

```

    ],
  ),
);
}
}

class GeneratorPage extends StatelessWidget {
  @override

```

```

Widget build(BuildContext context) {
  var appState = context.watch();
  var pair = appState.current;

  IconData icon;
  if (appState.favorites.contains(pair)) {
    icon = Icons.favorite;
  } else {
    icon = Icons.favorite_border;
  }
}

```

2.- Y se reemplazó por una estructura nueva basada en Row, la cual permite dividir la pantalla en dos secciones: navegación y contenido principal.

El código base quedó de la siguiente manera:

```

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [

```

```

        NavigationRailDestination(
            icon: Icon(Icons.home),
            label: Text('Home'),
        ),
        NavigationRailDestination(
            icon: Icon(Icons.favorite),
            label: Text('Favorites'),
        ),
    ],
    selectedIndex: 0,
    onDestinationSelected: (value) {
        print('selected: $value');
    },
),
),
Expanded(
    child: Container(
        color: Theme.of(context).colorScheme.primaryContainer,
        child: GeneratorPage(),
    ),
),
],
),
);
}

class GeneratorPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    IconData icon;
    if (appState.favorites.contains(pair)) {
      icon = Icons.favorite;
    } else {
      icon = Icons.favorite_border;
    }
  }
}

```

```

return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      BigCard(pair: pair),
      SizedBox(height: 10),
      Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          ElevatedButton.icon(
            onPressed: () {
              appState.toggleFavorite();
            },
            icon: Icon(icon),

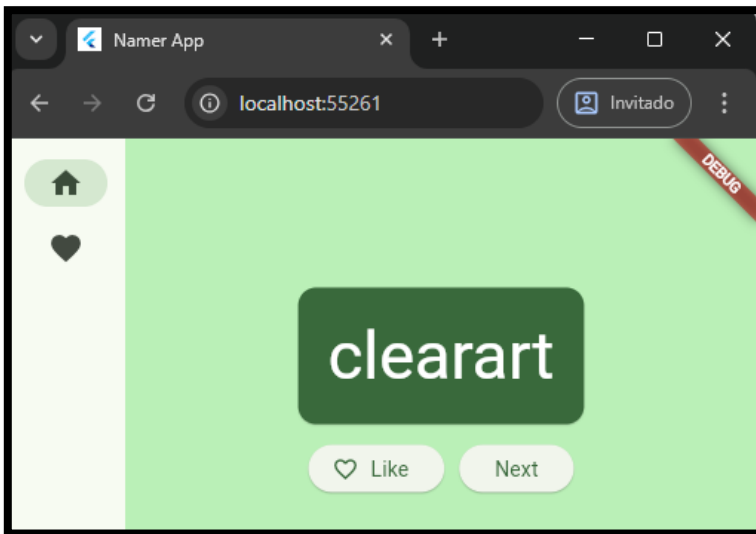
```

```

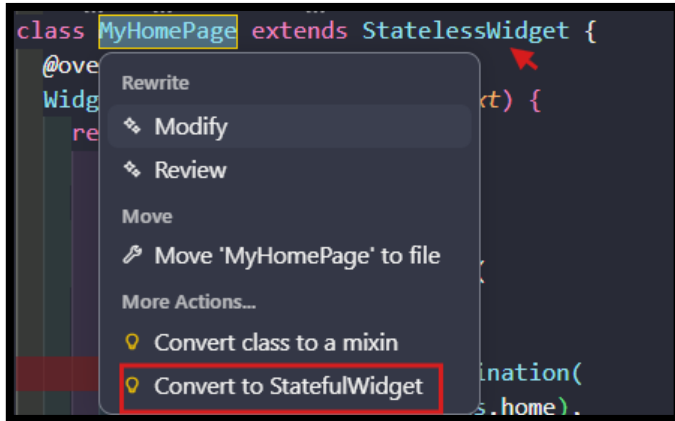
        label: Text('Like'),
      ),
      SizedBox(width: 10),
      ElevatedButton(
        onPressed: () {
          appState.getNext();
        },
        child: Text('Next'),
      ),
    ),
  ),
),
),
);
}

```

Al guardar los cambios, se puede observar que la parte visual de la interfaz ya está construida, sin embargo, aún no tiene funcionalidad. Por ejemplo, al presionar el ícono de corazón (♥) en el riel de navegación, no se genera ninguna acción ni cambio en la pantalla.



3.- Inicialmente, **MyHomePage** era un **StatelessWidget**. Se colocó el cursor en su definición y se accedió al menú **Refactor**, donde se seleccionó la opción **Convert to StatefulWidget**. Con esto, se generó automáticamente la clase **MyHomePageState**, permitiendo que el widget pueda actualizar su estado de forma dinámica.



```
class MyHomePage extends StatefulWidget {  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

4.- Se realizaron tres modificaciones dentro de la clase **MyHomePageState** para controlar la navegación de la aplicación:

1. Se declaró la variable de estado: `var selectedIndex = 0;`

Esta variable se inicializa en 0 y permite almacenar la opción actualmente seleccionada en el riel de navegación.

```
class MyHomePageState extends State<MyHomePage> {  
  var selectedIndex = 0;  
  @override  
  Widget build(BuildContext context) {
```

2. Se reemplazó el valor fijo del índice seleccionado por la variable: `selectedIndex`,

Con esto, la interfaz ahora depende del valor dinámico del estado.

3. Se actualizó el valor de `selectedIndex` cuando el usuario selecciona una opción:

```
setState(() {  
  selectedIndex = value;  
});
```

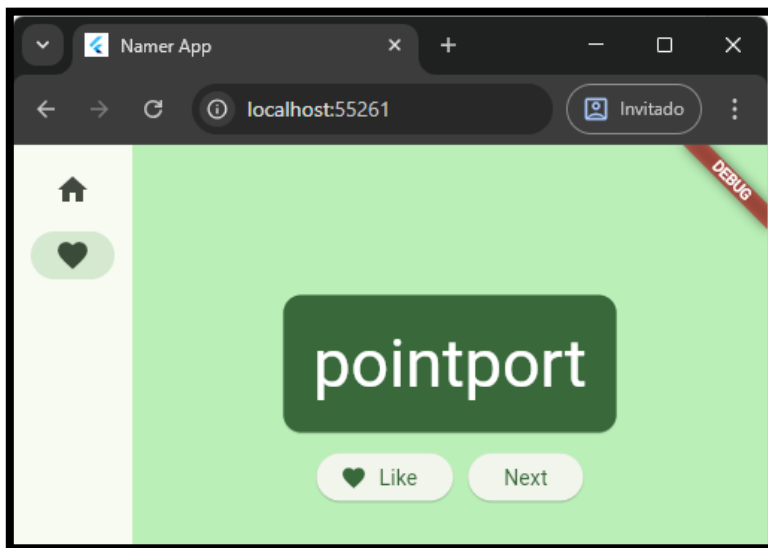
Esto permite que la interfaz se reconstruya automáticamente cada vez que cambia la selección, reflejando los cambios en pantalla.

```

        icon: Icon(Icons.favorite),
        label: Text('Favorites'),
      ), // NavigationRailDestination
    ],
    selectedIndex: selectedIndex,
    onDestinationSelected: (value) {
      setState(() {
        selectedIndex = value;
      });
    },
  },

```

La navegación ahora es dinámica y cambia de pantalla según la opción seleccionada en el riel.



5.- Se agregó una variable llamada `page`, la cual se encarga de almacenar el widget que se mostrará en pantalla dependiendo de la opción seleccionada en el riel de navegación.

Con este código, se utiliza una estructura `switch` para decidir qué pantalla mostrar:

- Si `selectedIndex` es 0, se muestra la pantalla principal `GeneratorPage`.
- Si es 1, se muestra un `Placeholder` como vista temporal.
- Si el valor no coincide con ninguno de los casos, se genera un error.

```

class _MyHomePageState extends State<MyHomePage> {
  var selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    Widget page;
    switch (selectedIndex) {
      case 0:
        page = GeneratorPage();
        break;
      case 1:
        page = Placeholder();
        break;
      default:
        throw UnimplementedError('no widget for $selectedIndex');
    }
  }
}

```

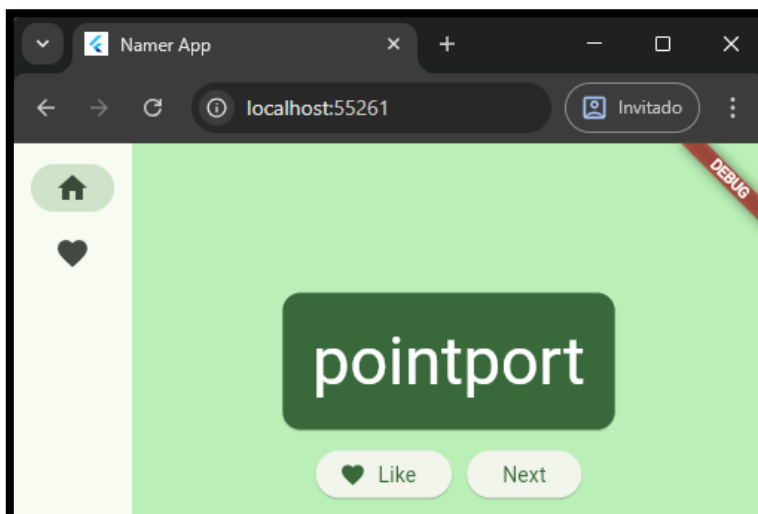
6.- Posteriormente, se reemplazó el contenido fijo del cuerpo por: **child: page**,
Con esto, la interfaz ahora muestra dinámicamente la pantalla correspondiente según la selección del usuario.

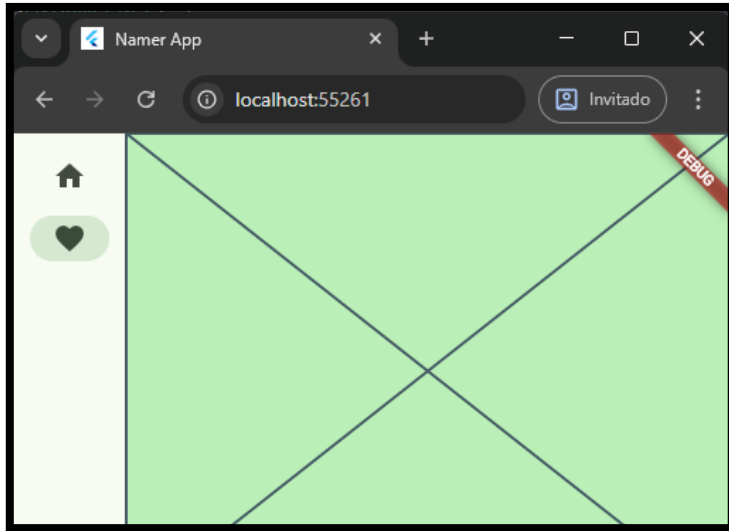
```

), // SafeArea
Expanded(
  child: Container(
    color: Theme.of(context).colorScheme.primaryContainer,
    child: page,
  ), // Container
), // Expanded

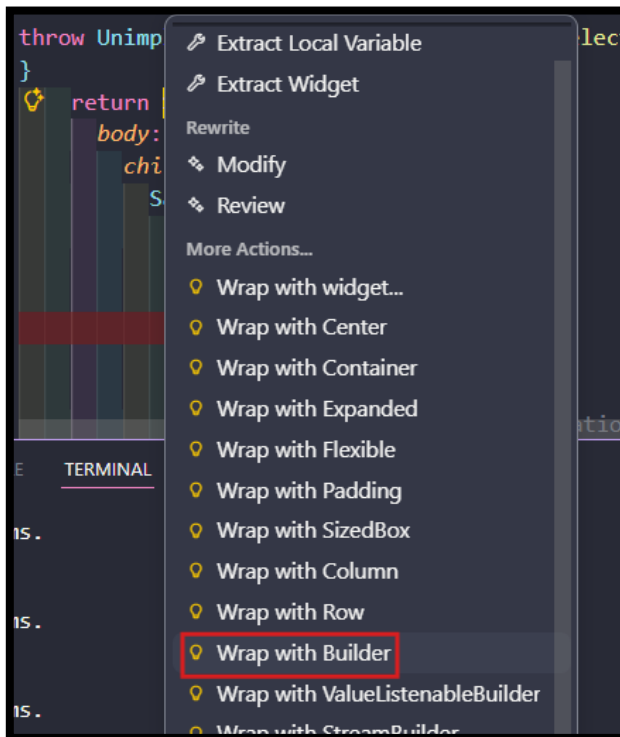
```

La aplicación ahora puede alternar entre la pantalla GeneratorPage y una vista temporal (Placeholder) que representa la futura sección de favoritos.





7.- Se colocó el cursor en el widget **Scaffold** dentro del método `build()` de `_MyHomePageState` y se accedió al menú **Refactor**, donde se seleccionó la opción **Wrap with Builder**.



8.- Posteriormente, el widget generado se renombró a `LayoutBuilder` y se modificó la función de retorno de `(context)` a `(context, constraints)` para poder acceder al tamaño disponible de la pantalla.

Sin modificar:

```

return Builder(
  builder: (context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [

```

Modificado:

```

return LayoutBuilder(
  builder: (context, constraints) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(

```

9.- Además, se actualizó la propiedad: **extended: constraints.maxWidth >= 600,**

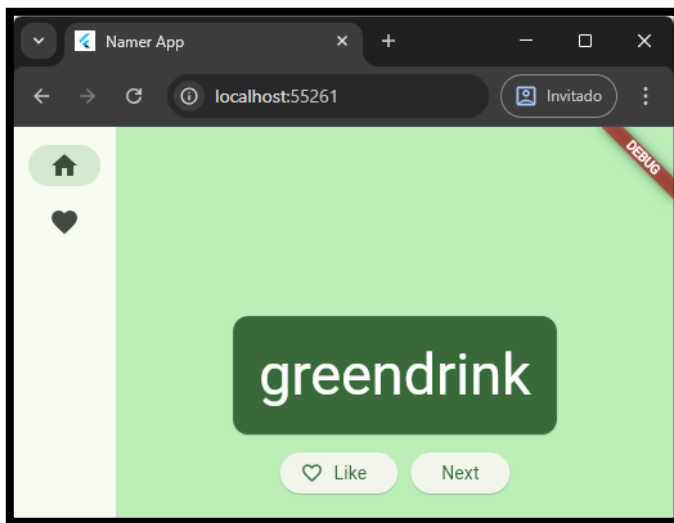
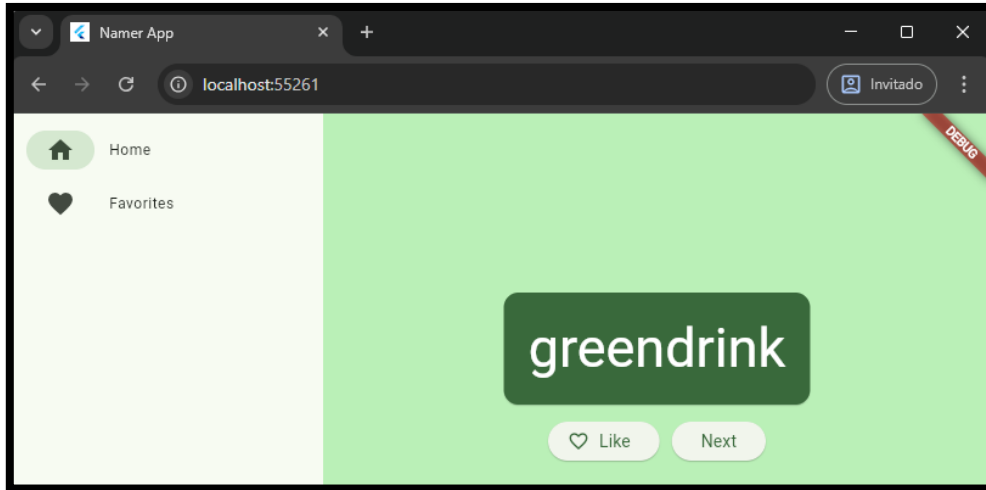
Reemplazando el valor fijo **extended: false**, para que ahora el riel de navegación se expanda automáticamente cuando haya suficiente espacio en pantalla.

```

return LayoutBuilder(
  builder: (context, constraints) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: constraints.maxWidth >= 600,
              destinations: [

```

Con esto, la aplicación puede adaptarse de forma dinámica al tamaño de la ventana o la orientación del dispositivo.



Parte 8

Creación de la página de favoritos

En esta etapa se tiene como objetivo reemplazar la pantalla temporal (Placeholder) por una nueva sección funcional destinada a mostrar los elementos marcados como favoritos dentro de la aplicación.

Se integra una nueva vista que permite gestionar y visualizar la información almacenada por el usuario, utilizando el estado global de la aplicación para acceder a los datos correspondientes.

Finalmente, se reemplaza el Placeholder en el sistema de navegación por la nueva pantalla de favoritos, completando así la estructura básica de la aplicación.

Para ello, inicialmente la lógica de navegación se encontraba definida de la siguiente manera:

case 1:

page = Placeholder();

1.- Posteriormente, este fragmento fue modificado para integrar la nueva pantalla de favoritos:

case 1:

page = FavoritesPage();



```
@override
Widget build(BuildContext context) {
  Widget page;
  switch (selectedIndex) {
    case 0:
      page = GeneratorPage();
      break;
    case 1:
      page = FavoritesPage();
      break;
    default:
      throw UnimplementedError('No widget for $selectedIndex');
  }
  return page;
}
```

2.- Después de este cambio, se creó una nueva clase llamada **FavoritesPage**, la cual se encarga de construir toda la interfaz de la pantalla de favoritos.

Dentro de su método build, primero se obtiene el estado global de la aplicación mediante `context.watch<MyAppState>()`, lo que permite acceder a la lista de elementos guardados en favoritos. A partir de esta lista, la interfaz se adapta dinámicamente: si no existen elementos, se muestra un mensaje centrado indicando que aún no hay favoritos; en caso contrario, se construye una lista desplazable utilizando `ListView.builder`, donde cada elemento se representa mediante una tarjeta (Card) con información del par de palabras, ícono de favorito y opciones de interacción como eliminar elementos. Esta estructura permite una visualización ordenada y dinámica de los datos.

Así quedó el código agregado:

```

class FavoritesPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var favorites = appState.favorites;

    if (favorites.isEmpty) {
      return Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(Icons.favorite_border, size: 60),
            SizedBox(height: 10),
            Text(
              'No favorites yet ❤️',
              style: Theme.of(context).textTheme.titleMedium,
            ), // Text
          ],
        ), // Column
      ); // Center
    }
  }
}

```

```

return Column(
  children: [
    Padding(
      padding: const EdgeInsets.all(16),
      child: Text(
        'You have ${favorites.length} favorites ❤️',
        style: Theme.of(context).textTheme.titleLarge,
      ), // Text
    ), // Padding

    Expanded(
      child: ListView.builder(
        itemCount: favorites.length,
        itemBuilder: (context, index) {
          var pair = favorites[index];

          return Card(
            margin: EdgeInsets.symmetric(horizontal: 10, vertical: 5),
            child: ListTile(
              leading: Icon(Icons.favorite, color: Colors.red),

              title: Text(
                pair.asLowerCase,
                style: TextStyle(fontWeight: FontWeight.bold),
              ), // Text

              subtitle: Text(
                "${pair.first} ${pair.second}",
              ), // Text
            ),
          );
        },
      ),
    ),
  ],
)

```

```

        trailing: Icon(Icons.delete),
        onTap: () {
          appState.removeFavorite(pair);
        },
      ), // ListTile
    ); // Card
  },
), // ListView.builder
), // Expanded
],
); // Column
}
}

```

3.- Para complementar esta funcionalidad, se agregó el método encargado de eliminar elementos de la lista de favoritos:

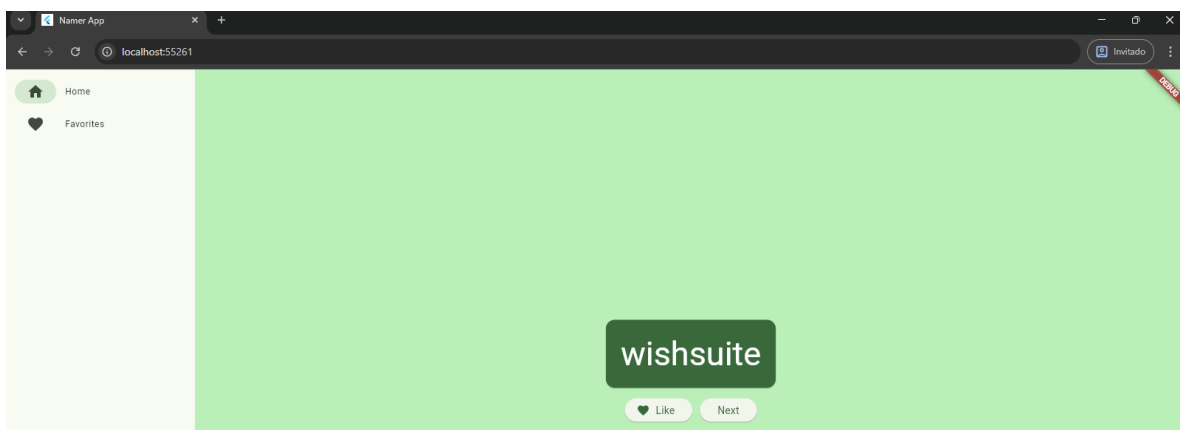
Este método permite actualizar automáticamente el estado de la aplicación, asegurando que la interfaz se reconstruya en tiempo real cada vez que un elemento es eliminado de favoritos.

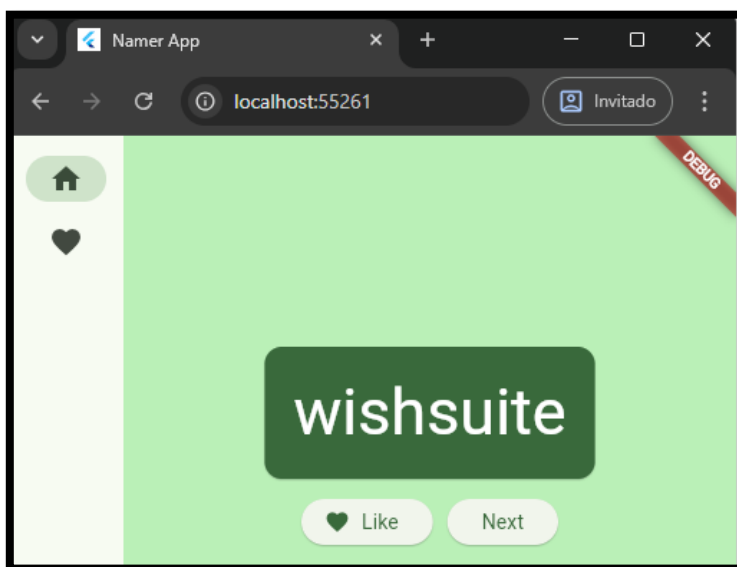
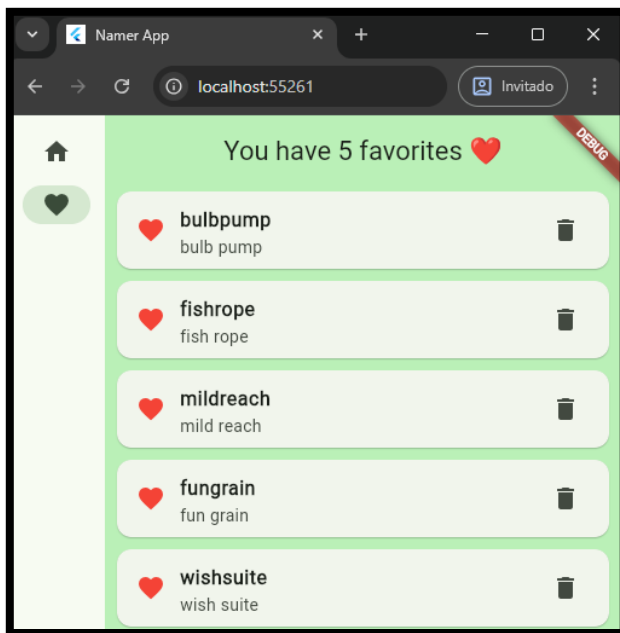
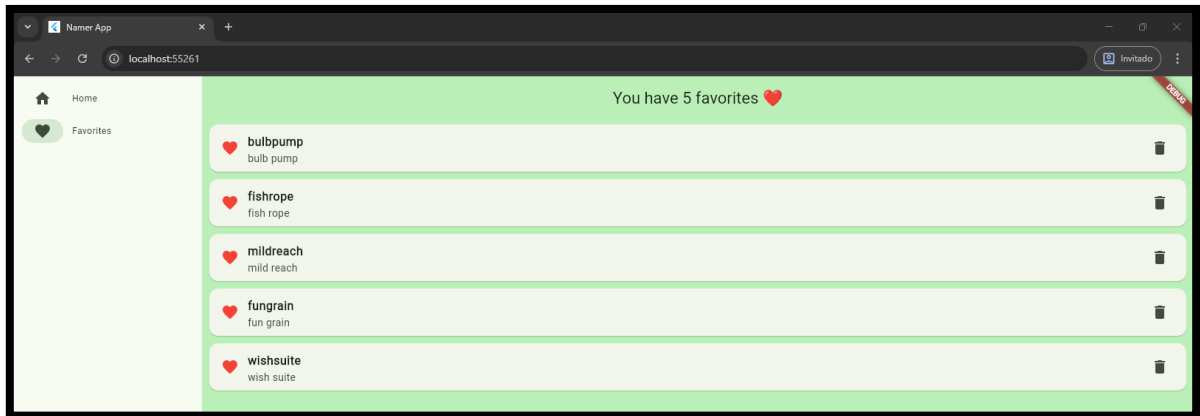
```

void removeFavorite(WordPair pair) {
  favorites.remove(pair);
  notifyListeners();
}

```

La aplicación ahora cuenta con una pantalla de favoritos completamente funcional, donde el usuario puede visualizar los elementos guardados en una lista dinámica, así como eliminarlos cuando lo desee. Además, la navegación entre pantallas se encuentra integrada correctamente, permitiendo alternar entre la generación de palabras y la sección de favoritos de manera fluida.





Nota: Durante el desarrollo de la práctica, no fue posible utilizar el emulador de Android Studio debido a limitaciones de espacio en disco. La instalación de las imágenes del sistema requería una gran cantidad de memoria, lo que generaba errores y provocaba un rendimiento lento en la computadora.

Por esta razón, se optó por ejecutar la aplicación en el navegador mediante el modo web, lo cual permitió continuar con el desarrollo sin afectar el rendimiento de la computadora.

Conclusión

En esta práctica se logró desarrollar una aplicación funcional utilizando Flutter, aplicando conceptos fundamentales como la creación de interfaces, manejo del estado, interacción con el usuario y navegación entre pantallas. A lo largo del proceso, se mejoró progresivamente tanto la estructura del código como la apariencia visual de la aplicación, logrando una interfaz más atractiva y organizada.

Además, se implementaron funcionalidades importantes como el almacenamiento de elementos favoritos y su visualización en una pantalla independiente, lo que permitió comprender mejor la gestión de datos dentro de una aplicación. Finalmente, se trabajó con diseño responsivo, permitiendo que la aplicación se adapte a diferentes tamaños de pantalla.